

UNIVERSIDAD TECNICA ESTATAL DE QUEVEDO
FACULTAD DE CIENCIAS DE LA COMPUTACION



**Practica: Fragmentacion de Base de Datos
Distribuida**

Materia: Aplicaciones Distribuidas (ISR-701)

Estudiante: Pedro Leonardo Castro Lopez

Semestre / Paralelo: 7MO

Docente: ING.GUERRERO ULLOA GLEISTON CICERON

Fecha: 07/07/2026

Índice

1. Introduccion	2
2. Objetivos	2
3. Arquitectura del sistema	2
4. Esquema central	3
5. Fragmentacion horizontal	3
6. Fragmentacion vertical	4
7. Fragmentacion mixta	4
8. Vistas globales con postgres_fdw	5
9. Verificacion	5
10.Conclusiones	6

1. Introduccion

La fragmentacion de bases de datos distribuidas permite dividir la informacion de un sistema centralizado en fragmentos que se ubican en distintos nodos de una red, con el objetivo de mejorar el rendimiento, la disponibilidad y la localidad de los datos. En esta practica se implementa un escenario de una cadena de cafeterias con tres sedes (Campus, Babahoyo y Ventanas), aplicando fragmentacion horizontal, vertical y mixta sobre una base de datos PostgreSQL desplegada en contenedores Docker.

2. Objetivos

- Diseñar e implementar un esquema de base de datos distribuida sobre tres nodos independientes de PostgreSQL.
- Aplicar fragmentacion horizontal sobre la tabla `pedidos` segun la sede de cada pedido.
- Aplicar fragmentacion vertical sobre la tabla `clientes` separando datos publicos de datos de contacto.
- Aplicar fragmentacion mixta combinando el criterio horizontal (ciudad) y vertical (sensibilidad de datos) sobre `clientes`.
- Reconstruir la informacion distribuida mediante vistas globales usando `postgres_fdw`.
- Verificar completitud, reconstruccion y disyuncion de los fragmentos generados.

3. Arquitectura del sistema

El sistema esta compuesto por tres nodos PostgreSQL 16, cada uno en un contenedor Docker independiente, sobre la base de datos `cafeteria`.

Nodo	Contenedor	Puerto host	Rol
Campus	pg-campus	5433	Nodo coordinador / fragmentos
Babahoyo	pg-babahoyo	5434	Fragmento Babahoyo
Ventanas	pg-ventanas	5435	Fragmento Ventanas

Cuadro 1: Distribucion de nodos

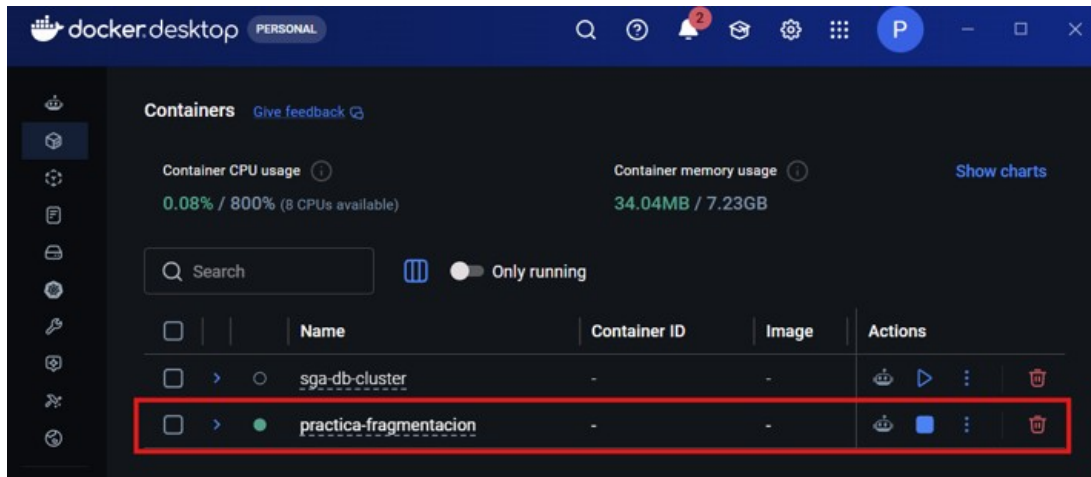


Figura 1: Arquitectura de la base de datos distribuida

4. Esquema central

Sobre pg-campus se crea el esquema centralizado inicial, con las tablas `clientes`, `productos` y `pedidos`, y se cargan los datos base (6 clientes, 5 productos, 8 pedidos).

```
CREATE TABLE
CREATE TABLE
PS C:\Users\leonardo\Desktop\practica-fragmentacion> docker exec -it pg-campus psql -U admin -d cafeteria -c "\dt"
List of relations
Schema | Name      | Type  | Owner
-----+-----+-----+-----
public | clientes  | table | admin
public | pedidos   | table | admin
public | productos | table | admin
(3 rows)
```

Figura 2: Creacion del esquema central y carga de datos

5. Fragmentacion horizontal

La tabla `pedidos` se fragmenta por el atributo `sede`: cada nodo conserva unicamente los pedidos generados en su propia sede.

Nodo	Sede	pedido_id
pg-campus	Campus	1, 3, 7
pg-babahoyo	Babahoyo	2, 6, 8
pg-ventanas	Ventanas	4, 5

Cuadro 2: Fragmentacion horizontal de pedidos

```

Terminal Local x + v
PS C:\Users\leonardo\Desktop\practica-fragmentacion> docker exec pg-campus psql -U admin -d cafeteria -c "\dt"cls
psql: warning: extra command-line argument "cls" ignored
List of relations
Schema | Name | Type | Owner
-----+-----+-----+-----
public | clientes | table | admin
public | clientes_mixta_quevedo_publicos | table | admin
public | clientes_publicos | table | admin
public | pedidos | table | admin
public | pedidos_centralizado_original | table | admin
public | productos | table | admin
(6 rows)

PS C:\Users\leonardo\Desktop\practica-fragmentacion> docker exec pg-campus psql -U admin -d cafeteria -c "SELECT * FROM pedidos ORDER BY pedido_id;"
pedido_id | cliente_id | producto_id | fecha | monto | sede
-----+-----+-----+-----+-----+-----
1 | 1 | 1 | 2026-03-01 | 0.75 | Campus
3 | 3 | 2 | 2026-03-02 | 1.00 | Campus
7 | 1 | 3 | 2026-03-04 | 2.50 | Campus
(3 rows)

PS C:\Users\leonardo\Desktop\practica-fragmentacion> docker exec pg-babahoyo psql -U admin -d cafeteria -c "SELECT * FROM pedidos ORDER BY pedido_id;"
pedido_id | cliente_id | producto_id | fecha | monto | sede
-----+-----+-----+-----+-----+-----
2 | 2 | 1 | 2026-03-01 | 0.50 | Babahoyo

```

Figura 3: Verificación de la fragmentación horizontal en cada nodo

6. Fragmentación vertical

La tabla `clientes` se divide por sensibilidad de la información: los datos públicos (nombre, ciudad) se almacenan en `pg-campus` como `clientes_publicos`, y los datos de contacto (email, teléfono) se almacenan en `pg-babahoyo` como `clientes_contacto`. El `cliente_id` se repite en ambos fragmentos para permitir la reconstrucción mediante JOIN.

```

PS C:\Users\leonardo\Desktop\practica-fragmentacion> docker exec pg-campus psql -U admin -d cafeteria
quevedo_publicos;"
cliente_id | nombre | ciudad
-----+-----+-----
1 | María Alvarado | Quevedo
3 | Ana Vera | Quevedo
(2 rows)

PS C:\Users\leonardo\Desktop\practica-fragmentacion> docker exec pg-ventanas psql -U admin -d cafeteria
a_bv_publicos;"
cliente_id | nombre | ciudad
-----+-----+-----
2 | Luis Ceden | Babahoyo
4 | Jose Mendoza | Ventanas
5 | Carla Zambrano | Ventanas
6 | Pedro Suarez | Babahoyo
(4 rows)

```

Figura 4: Fragmentación vertical de clientes

7. Fragmentación mixta

Se combina el corte horizontal por ciudad con el corte vertical por sensibilidad de datos:

Ciudad	Corte vertical	Columnas	Nodo
Quevedo	publicos	cliente_id, nombre, ciudad	pg-campus
Quevedo	contacto	cliente_id, email, telefono	pg-babahoyo
Babahoyo/Ventanas	publicos	cliente_id, nombre, ciudad	pg-ventanas
Babahoyo/Ventanas	contacto	cliente_id, email, telefono	pg-babahoyo

Cuadro 3: Fragmentacion mixta de clientes

```
PS C:\Users\leonardo\Desktop\practica-fragmentacion> docker exec pg-babahoyo psql -U admin -d cafeteria -c "SELECT * FROM clientes_mixta_contacto;"cls
psql: warning: extra command-line argument "cls" ignored
 cliente_id | email | telefono
-----
1 | maria@uteq.edu.ec | 0991111111
2 | luis@uteq.edu.ec | 0992222222
3 | ana@uteq.edu.ec | 0993333333
4 | jose@uteq.edu.ec | 0994444444
5 | carla@uteq.edu.ec | 0995555555
6 | pedro@uteq.edu.ec | 0996666666
(6 rows)

PS C:\Users\leonardo\Desktop\practica-fragmentacion> docker exec pg-campus psql -U admin -d cafeteria -c "\dv"
List of relations
Schema | Name | Type | Owner
-----
public | clientes_global | view | admin
public | pedidos_global | view | admin
(2 rows)

PS C:\Users\leonardo\Desktop\practica-fragmentacion> docker exec pg-campus psql -U admin -d cafeteria -c "\det"
List of foreign tables
Schema | Table | Server
-----
public | clientes_contacto_fdw | server_babahoyo
public | pedidos_babahoyo | server_babahoyo
public | pedidos_ventanas | server_ventanas
(3 rows)
```

Figura 5: Fragmentacion mixta aplicada en los tres nodos

8. Vistas globales con postgres_fdw

Sobre pg-campus se configura la extension postgres_fdw para acceder a los fragmentos remotos de pg-babahoyo y pg-ventanas. Se crean foreign tables para los fragmentos de pedidos y para clientes_contacto, y a partir de estas se construyen dos vistas:

- pedidos_global: union de los tres fragmentos horizontales.
- clientes_global: join entre clientes_publicos y el fragmento remoto clientes_contacto_fdw.

```
PS C:\Users\leonardo\Desktop\practica-fragmentacion> docker exec pg-campus psql -U admin -d cafeteria -c "\det"
List of foreign tables
Schema | Table | Server
-----
public | clientes_contacto_fdw | server_babahoyo
public | pedidos_babahoyo | server_babahoyo
public | pedidos_ventanas | server_ventanas
(3 rows)

PS C:\Users\leonardo\Desktop\practica-fragmentacion> docker exec pg-campus psql -U admin -d cafeteria -c "SELECT * FROM pedidos_global ORDER BY pedido_id;"cls
psql: warning: extra command-line argument "cls" ignored
 pedido_id | cliente_id | producto_id | fecha | monto | sede
-----
1 | 1 | 1 | 2026-03-01 | 0.75 | Campus
2 | 2 | 2 | 2026-03-01 | 2.50 | Babahoyo
3 | 3 | 3 | 2026-03-02 | 1.00 | Campus
4 | 4 | 4 | 2026-03-02 | 1.25 | Ventanas
5 | 5 | 5 | 2026-03-03 | 1.00 | Ventanas
6 | 6 | 1 | 2026-03-03 | 0.75 | Babahoyo
7 | 7 | 3 | 2026-03-04 | 2.50 | Campus
8 | 8 | 2 | 2026-03-04 | 1.00 | Babahoyo
(8 rows)
```

Figura 6: Creacion de servidores foraneos, foreign tables y vistas globales

9. Verificacion

Se ejecutan tres consultas de comprobacion sobre pedidos_global:

1. **Completitud:** el total de filas de `pedidos_global` debe ser igual al total original (8 pedidos).
2. **Reconstruccion:** la suma de monto agrupada por `sede` en la vista global debe coincidir con el mismo calculo sobre la copia centralizada original.
3. **Disyuncion:** ningun `pedido_id` debe repetirse entre los fragmentos.

```

Terminal Local x v
PS C:\Users\leonardo\Desktop\practica-fragmentacion> Get-Content sql\07_verificacion.sql | docker exec -i pg-campus psql -U admin -d cafeteria
total_pedidos_global
-----
      8
(1 row)

 sede | total_monto
-----+-----
 Babahoyo |      4.25
  Campus |      4.25
 Ventanas |      2.25
(3 rows)

 sede | total_monto
-----+-----
 Babahoyo |      4.25
  Campus |      4.25
 Ventanas |      2.25
(3 rows)

 pedido_id | count
-----+-----
(0 rows)

```

Figura 7: Resultados de las consultas de verificacion

Los resultados obtenidos confirman que la fragmentacion cumple las tres propiedades: 8 filas totales en la vista global, montos por sede iguales en ambas fuentes (Babahoyo 4.25, Campus 4.25, Ventanas 2.25), y cero filas con `pedido_id` repetido.

10. Conclusiones

- La fragmentacion horizontal permitio distribuir los pedidos segun la sede donde se originan, reduciendo la cantidad de datos que cada nodo debe almacenar y acercando la informacion al lugar donde se genera.
- La fragmentacion vertical permitio separar datos sensibles (contacto) de datos publicos, lo que facilita aplicar politicas de acceso distintas a cada fragmento.
- La combinacion de ambos criterios en la fragmentacion mixta demuestra que es posible aplicar mas de una estrategia sobre la misma tabla segun las necesidades del sistema.
- El uso de `postgres_fdw` permitio reconstruir una vista unificada de los datos distribuidos sin necesidad de replicar la informacion completa en un solo nodo.
- Las consultas de verificacion confirmaron que la fragmentacion es completa, reconstruible y disjunta, cumpliendo las tres propiedades fundamentales de un esquema de fragmentacion correcto.